

CVE-2026-31431 — Copy Fail

Linux Kernel Local Privilege Escalation • Disclosed 2026-04-29 • CVSS 7.8 HIGH • All major distros affected

9 yr

Exposure window (2017–2026)

732 B

Exploit size (Python)

<1 sec

Time to root

0

Forensic artifacts left

WHY THIS IS NOT A NORMAL CVE

No race condition. No KASLR bypass. No kernel address leak. Deterministic, 100% reliable, runs as any unprivileged user, leaves no trace on disk or in audit logs. Same class as Dirty Pipe (CVE-2022-0847) but with better operational security built into the physics of the bug.

What's Broken

Three kernel components that were never designed to interact:

Component	Shipped	Purpose
<code>authencesn</code> algorithm	2011	IPsec ESP extended sequence number authentication
<code>AF_ALG</code> AEAD socket support	2015	Exposes kernel crypto operations to unprivileged userspace
In-place optimization in <code>algif_aead.c</code>	2017 (commit <code>72548b093ee3</code>)	Performance optimization merging source/destination scatterlists

The 2017 optimization set `req->src = req->dst` , meaning AEAD operations read and write to the same memory buffers. The `authencesn` algorithm writes 4 scratch bytes at a controlled offset as internal working space. When a user `splice()` s a file’s page-cache pages into the crypto pipeline, those scratch bytes land **directly in the page cache of the target file** – bypassing all file permission checks.

Attack Chain

- 1 Open an `AF_ALG` socket and bind to `authencesn(hmac(sha256),cbc(aes))` . No privileges required.
- 2 `splice()` a target file’s page-cache pages into the crypto socket via a pipe. The kernel maps the actual page-cache pages by reference, not by copy.
- 3 Controlled 4 bytes are placed in the AAD (Associated Authenticated Data) as `seqno_lo` . The `authencesn` algorithm writes these 4 bytes into the destination scatterlist at a calculated offset – which is the file’s page cache.
- 4 The HMAC check fails (garbage ciphertext), `recv()` returns an error – but the scratch write has already landed. Iterate to build up arbitrary page-cache modifications 4 bytes at a time.
- 5 Execute the corrupted binary or leverage the corrupted data. Root achieved.

TWO KNOWN EXPLOITATION STRATEGIES

Binary corruption: Write compressed shellcode into the page-cached copy of `/usr/bin/su`, then execute it.

UID patching: Overwrite the running user's UID field in the page-cached `/etc/passwd` from `1000` to `0000`, then `su <user>` with your own password drops a root shell.

Why There Is No Forensic Trail

- **Page cache is volatile memory.** The corruption lives in RAM, not on disk. No file modification timestamp, no journal entry, no `fsck` evidence.
- **The kernel never marks the page dirty.** The write came through the crypto subsystem, not the filesystem write path – so it is never written back to disk.
- **Reboot destroys all evidence.** Cold boot = clean state. Memory pressure evicts the page silently.
- **auditd doesn't see it.** There is no `write()` or `open(..., O_WRONLY)` syscall to log.
- **File integrity tools see a clean file.** AIDE, Tripwire, `rpm -Va` all compare against disk.
- **inotify / fanotify don't fire.** The file was never "modified" from the VFS perspective.

Only real-time detection works: Falco or Sysdig watching for `socket(AF_ALG, SOCK_SEQPACKET) + bind("aead", "authencsn(...))`. After the fact, there is nothing to find.

Affected Systems

Every Linux kernel built with `CONFIG_CRYPTO_USER_API_AEAD` from kernel 4.14 (late 2017) **or later**, up to the patched versions.

Distribution	Status	Notes
Red Hat Enterprise Linux 10.1	CONFIRMED	Kernel 6.12.0 – <code>algif_aead</code> compiled in (<code>=y</code>), cannot unload
Red Hat Enterprise Linux 9.x	AFFECTED	Kernel 5.14+ – in vulnerable range
Red Hat Enterprise Linux 8.x	AFFECTED	Kernel 4.18 – in vulnerable range
Ubuntu 24.04 LTS	CONFIRMED	Exploit verified by researchers
Amazon Linux 2023	CONFIRMED	Kernel 6.18
SUSE 16	CONFIRMED	Kernel 6.12
Alpine Linux (containers)	AFFECTED	<code>algif_aead</code> shipped as module by default
NVIDIA L4T (Jetson)	AFFECTED	Ubuntu-based kernel, JetPack 5.x/6.x – see edge section below
OpenWrt / DD-WRT	LIKELY SAFE	Stripped kernels usually disable <code>AF_ALG</code> userspace API
Fedora (current)	PATCHED	Kernel $\geq$ 6.19.12

CVSS & Tracking

Field	Value
CVE	CVE-2026-31431
CVSS 3.1	7.8 HIGH
Attack Vector	Local
Attack Complexity	Low
Privileges Required	Low (any unprivileged user)
User Interaction	None
Red Hat Bugzilla	Bug 2460538 – Status: NEW, Severity: High
Upstream Fix	Commit <code>a664bf3d603d</code> (Herbert Xu) – kernels 7.0, 6.19.12, 6.18.22
Introduced	Commit <code>72548b093ee3</code> – kernel 4.14 (2017)
Disclosed	2026-04-29
Discovered By	Theori / Xint Code (AI-powered security research) – found in ~1 hour

Immediate Mitigation

If `algif_aead` is a loadable module

```
# Blacklist the vulnerable module
echo "install algif_aead /bin/false" > /etc/modprobe.d/disable-algif-aead.conf
rmmod algif_aead 2>/dev/null

# Rebuild initramfs
dracut -f           # RHEL / Fedora
update-initramfs -u # Ubuntu / Debian
```

If `algif_aead` is compiled in (`=y`)

Module blacklisting **does not work**. The only mitigations are:

- **Kernel patch** – update to a fixed kernel version
- **SELinux custom policy** – RHEL's default targeted policy allows `alg_socket` creation, but a single custom module can deny it system-wide. AppArmor can deny `AF_ALG` per-profile (`deny network alg,`) but requires updating each profile individually. See mitigation section below.
- **seccomp** – block `socket(AF_ALG, ...)` in container runtimes

WHAT THE MODULE BLACKLIST DOES NOT BREAK

dm-crypt / LUKS, kTLS, IPsec / XFRM, OpenSSL, GnuTLS, NSS, SSH. These all use kernel crypto directly without the `AF_ALG` userspace socket interface.

RHEL 10.1 NOTE

On RHEL 10.1 (kernel 6.12.0-124.x), `algif_aead` is compiled into the kernel (`CONFIG_CRYPTO_USER_API_AEAD=y`). It cannot be unloaded or blacklisted. You must wait for the kernel errata or deploy SELinux/seccomp mitigations. Red Hat Bugzilla 2460538 is still in NEW status as of 2026-04-30.

Risk by Environment

Environment	Risk	Impact
Multi-tenant servers (shared shell, dev boxes)	CRITICAL	Any tenant gets root, compromises all tenants
Kubernetes / container platforms	CRITICAL	Container escape via shared page cache; node-level root
CI/CD runners (GitHub Actions, GitLab, Jenkins)	CRITICAL	Untrusted pipeline code gets host root
AI/ML inference endpoints	CRITICAL	Model-serving containers escape to host
Edge / IoT devices (Jetson, industrial)	CRITICAL	LPE + no monitoring + slow patch cycles = persistent compromise
Single-tenant production servers	HIGH	LPE chaining vector after initial foothold
Single-user workstations	MODERATE	Privilege escalation after initial compromise

Copy Fail at the Edge: NVIDIA Jetson, Ubuntu, and the OT Attack Surface

The “local only” attack vector in the CVSS score masks the real-world risk at the operational technology (OT) edge. Edge devices running mainstream Linux kernels are fully exposed – and they operate in environments where the standard mitigations do not exist.

NVIDIA Jetson Is Vulnerable

NVIDIA Jetson platforms (Xavier, Orin) run **L4T (Linux for Tegra)** – a full Ubuntu kernel with NVIDIA GPU and DLA drivers. The kernel is not stripped down.

JetPack Version	Kernel	Base OS	Status
JetPack 6.x (Orin)	5.15 / 6.1	Ubuntu 22.04	VULNERABLE
JetPack 5.x (Xavier / Orin)	5.10	Ubuntu 20.04	VULNERABLE
JetPack 4.x (Nano / TX2)	4.9	Ubuntu 18.04	BELOW RANGE (kernel 4.9 < 4.14)

`CONFIG_CRYPTO_USER_API_AEAD` is enabled in L4T’s default kernel config. NVIDIA needs the crypto subsystem for dm-crypt, container runtime support, and the network stack. The attack surface is present.

Why Edge Devices Are Worse Than Servers

THE COMPOUNDING FACTORS AT THE EDGE

- **Patch lag is measured in months, not days.** NVIDIA controls the L4T kernel. Customers cannot `dnf update` – they must wait for NVIDIA to ship a patched JetPack BSP, then requalify it against their workloads, then roll it to potentially hundreds of field devices.
- **No runtime security monitoring.** Nobody is running Falco, Sysdig, or RHACS on a Jetson in a signal bungalow or a wayside cabinet. The only detection method for Copy Fail (watching for `AF_ALG` socket creation) does not exist at the edge.
- **Ubuntu ships AppArmor, not SELinux.** AppArmor can deny `AF_ALG` via `deny network alg,` rules, but each application profile must be updated individually. SELinux deploys a single module system-wide – a significant operational advantage when hardening hundreds of edge devices.
- **“Local” access is more achievable than it sounds.** Exposed SSH, vulnerable inference APIs, compromised containers, physical access to field-deployed hardware – any foothold becomes root in under a second.
- **Containerized inference is the NVIDIA-recommended pattern.** DeepStream, Triton, and custom models typically run in containers on Jetson. Copy Fail jumps from container to host via the shared page cache. Container isolation is meaningless.

What These Devices Are Doing

NVIDIA Jetson is deployed across critical infrastructure for edge AI inference:

Industry	Edge AI Use Cases	Consequence of Root Compromise
Rail / Transportation	Track defect detection, wayside monitoring, container ID scanning, PTC edge compute	Safety system manipulation, signal interference, surveillance of operations
Utilities / Energy	Substation monitoring, predictive maintenance, grid edge analytics	SCADA/ICS lateral movement, grid visibility disruption
Manufacturing	Quality inspection, robotic guidance, process control	Production sabotage, IP exfiltration, safety system bypass
Healthcare	Medical imaging inference, patient monitoring, pharmacy automation	HIPAA breach, patient safety risk, operational disruption
Retail / Food	Refrigeration monitoring, loss prevention, self-checkout AI	PCI DSS scope, food safety monitoring gaps

The IT/OT Convergence Problem

Edge AI devices are the convergence point between IT and OT networks. A Jetson running computer vision on the OT side is also pulling model updates, reporting telemetry, and running containers – all of which require IP connectivity that creates attack surface. Copy Fail turns any foothold on these devices into root with no forensic evidence.

This is not a hypothetical chain. The components are:

1. Exposed service or container vulnerability on the Jetson → unprivileged shell
2. Copy Fail → root in <1 second, zero artifacts
3. Root on an OT-connected device → lateral movement into industrial control systems
4. No runtime detection at the edge → persistence until hardware replacement

The Red Hat Counter-Position

Exposure	Ubuntu on Jetson (Current State)	Red Hat Device Edge on Jetson
Kernel patching	Wait for NVIDIA JetPack BSP release	Red Hat kernel errata on Red Hat's cadence, independent of NVIDIA
Update mechanism	<code>apt upgrade</code> – mutable filesystem, no rollback	<code>bootc</code> image-based updates – atomic, rollback-capable
MAC enforcement	AppArmor – per-profile <code>deny network alg,</code> (no system-wide deny)	SELinux – single custom module denies <code>AF_ALG</code> system-wide
Runtime detection	None (no Falco, no Sysdig)	Red Hat Advanced Cluster Security (RHACS) with seccomp + Falco integration
Container isolation	Docker – no seccomp filter for <code>AF_ALG</code>	Podman + custom seccomp policy blocking <code>AF_ALG</code> socket creation
Compliance posture	No FIPS, no CC evaluation	FIPS 140-3 validated crypto modules, STIG content, CC certification

SELINUX: THE MECHANISM EXISTS, BUT REQUIRES CUSTOM POLICY

RHEL's default targeted policy **allows** `alg_socket` creation from all confined domains (`user_t` , `container_t` , `staff_t`). The exploit works under default SELinux enforcing. However, SELinux provides the **mechanism** to block it: a single custom policy module denying `alg_socket { create }` applies system-wide to all confined domains in one operation. AppArmor can also deny `AF_ALG` sockets via `deny network alg,` rules, but each application profile must be updated individually – there is no system-wide equivalent. At fleet scale, the operational difference matters: one `semodule -i` vs. editing every profile on every host.

Custom SELinux module to block Copy Fail

```
# copyfail_deny.te
module copyfail_deny 1.0;
require { type user_t; type staff_t; type container_t; class alg_socket create; }
neverallow user_t    self:alg_socket create;
neverallow staff_t    self:alg_socket create;
neverallow container_t self:alg_socket create;
```

```
# Build and install:
checkmodule -M -m -o copyfail_deny.mod copyfail_deny.te
semodule_package -o copyfail_deny.pp -m copyfail_deny.mod
semodule -i copyfail_deny.pp
```

Talking Points

- **“Local only” on a CVSS score does not mean “safe at the edge.”** Any exposed service, any container escape, any physical access to field hardware satisfies the local vector requirement.
- **You cannot patch what you do not control.** When the OS kernel comes from the hardware vendor (NVIDIA L4T), your security posture is gated by their release schedule. Red Hat Device Edge decouples the kernel from the hardware vendor.
- **Container isolation is not a security boundary for this class of vulnerability.** The page cache is host-wide. Containerized workloads on Jetson share the page cache with the host. Namespace isolation provides zero protection.
- **Red Hat OpenShift at the edge closes this class of vulnerability.** SELinux, seccomp, RHACS, and `bootc` image-based updates provide defense in depth that the current Ubuntu-on-Jetson stack cannot match.
- **Nine years of exposure with zero detection capability.** How many edge devices were compromised via this or similar page-cache attacks? Nobody knows, because nobody was watching, and the exploit leaves no trace.

